

Einführung in die Programmierung (WIN+BIT)

Kontrollstrukturen: Exceptions

Prof. Dr. Johannes C. Schneider / Prof. Dr. Markus Eiglsperger



Laufzeit-Fehlerquellen

- Unter bestimmten Umständen kann ein Programm nicht ordnungsgemäß weitergeführt werden, sondern liefert einen Fehler und bricht ab.
- **Viele Fehlerquellen** sind denkbar:
 - Arithmetische Operationen (z.B. Division durch 0)
 - Zugriff auf Datei, die nicht existiert
 - Schreibzugriff auf schreibgeschützte Datei
 - Abbruch von Netzwerkverbindungen
 - Casting inkompatibler Typen
 - ...

Exceptions

- **Trennung** von regulärem Code und Fehlerbehandlung
- In **Ausnahmesituation** (z.B. Division durch 0, Dateizugriff-Fehler, keine Internetverbindung)
 - Erzeugung eines **Exception-Objekts**
 - Informationen zu Fehlerursache und –ort in Objekt
- Weitergabe von Exceptions **nicht über Methodenrückgabewerte**, sondern über speziellen Exception-Mechanismus
- Abruf der Fehlerinformationen über **Instanzmethoden** des Exception-Objekts

java.lang.Exception

- Exception-Objekt ist Instanz von `java.lang.Exception` (oder von einer Unterklasse)
- Fehlerursache abrufbar mittels
`public String getMessage()`
- Fehlerort abrufbar mittels
`public void printStackTrace()`



code20.
ShowStack
Trace

Abfangen von Exceptions

- In Ausnahmesituation wird Exception "**geworfen**" ("to throw an exception")
- Exceptions können **abgefangen** werden
 - Abstürze vermeiden → Kontrolle behalten
 - Fehlerbehandlung ausführen
 - Fehler ausgeben (falls hilfreich für Benutzer)
 - Arbeiten abschließen vor Programmende, z.B. Daten sichern, Ressourcen freigeben
 - Gültigen Zustand wiederherstellen, Operation erneut versuchen

Ein Methode wirft eine Exception

- Wenn eine Methode eine Exception definiert, muss man sie behandeln (Ausnahme: `RuntimeException`, s. später)
- Beispiel:

```
java.nio.file.Files  
public static String readString(@NotNull java.nio.file.Path path)  
throws java.io.IOException
```

Reads all content from a file into a string, decoding from bytes to characters using the `UTF-8 charset`. The method ensures that the file is closed when all content have been read or an I/O error, or other runtime exception, is thrown. This method is equivalent to: `readString(path, StandardCharsets.UTF_8)`

Params:	path – the path to the file
Returns:	a String containing the content read from the file
Throws:	<code>java.io.IOException</code> – if an I/O error occurs reading from the file or a malformed or unmappable byte sequence is read



code20.
Exceptions1

Exception-Handling: Weiterleiten

- Weiterleitung an den Aufrufer der Methode (via throws an der Methoden-Deklaration)

```
public static void main(String[] args) throws IOException {  
    // ...  
    String content = Files.readString(Paths.get(fileName));  
}
```

- Im Zweifel kann man die Exceptions so bis zur main()-Methode weiter"werfen", wo sie dann zum Programmabbruch mit Fehlermeldung sorgen

- Mehrere Exceptions weiterleiten:

```
void method(int arg) throws NoSuchElementException,  
    SocketException, UnsupportedEncodingException { ... }
```



code20.

Exceptions2a



code20.

Exceptions2b

Exception-Handling: Fangen

- Behandlung mit try-catch ("Abfangen von Exceptions"), dabei möglich

```
try {  
    content = Files.readString(Paths.get(fileName));  
    System.out.println("Zugriff hat geklappt!");  
} catch (IOException e) {  
    e.printStackTrace();  
    return; // main-Methode verlassen und Programm beenden  
}
```

Wird nur
ausgeführt, wenn
es vorher **keine**
Exception gab.



code20.
Exceptions3

Exception-Handling, Antipattern

- Exception einfach verschlucken

```
try {  
    content = Files.readString(Paths.get(fileName));  
} catch (IOException e) {  
    // wird schon nicht so schlimm sein  
}
```

- Exception auf der Kommandozeile loggen

```
try {  
    content = Files.readString(Paths.get(fileName));  
} catch (IOException e) {  
    System.out.println("Fehler: " + e); // Mal sehn, wer's liest  
}
```



code20.

Exceptions4

checked vs. unchecked

- Exceptions, die von `RuntimeException` erben, sind sog. "unchecked exceptions", die **man nicht behandeln muss** (aber kann) → Im Beispiel die `NullPointerException`
- unchecked exceptions verwendet man Situationen, in denen der Code, der die Exception "abbekommt", nicht sinnvoll auf eine Exception reagieren kann, weil jemand das System anpassen und neustarten muss, z.B.
 - Konfigurationsdatei nicht vorhanden
 - Programmierfehler, wenn eine Methode oder ein Konstruktor mit einem ungültigen Argument aufgerufen wurde.
- Viele Informationen zu Exceptions: <http://docs.oracle.com/javase/tutorial/essential/exceptions/index.html>



code20.

Exceptions5a/
Exceptions5b

Häufige RuntimeExceptions

- `NullPointerException`: Parameter ist unerwartet null
- `IllegalArgumentException`: Parameter ist nicht null, hat aber ungültigen Wert
- `IllegalStateException`: Der Zustand der aktuellen Klasse (des Systems) erlaubt den Methodenaufruf nicht (z.B. Aufruf von `Scanner#next`, nachdem `Scanner#close` aufgerufen wurde)
- `IndexOutOfBoundsException` (auch `StringIndex...` und `ArrayIndex...`): Index außerhalb des gültigen Bereichs
- `NoSuchElementException`: Element (z.B. einer Liste) nicht vorhanden
- `UnsupportedOperationException`: Methodenaufruf nicht unterstützt (z.B. bei Interface-Implementierungen, die nur einen Teil der Methoden implementieren)

catch: Behandeln unterschiedlicher Exceptions

- Bei der Behandlung mit try-catch ("Abfangen von Exceptions") sind mehrere catch-Blöcke möglich:

```
String fileName = ...;
try {
    printFirst100Chars(fileName);
} catch (NullPointerException e) {
    System.out.println("Mensch, man darf "
        + "printFirst100Chars doch nicht mit null aufrufen!");
} catch (IOException e) {
    System.out.println("Fehlermeldung: " + e.getMessage());
}
```

Disclaimer: Dieses Beispiel illustriert, wie man mehrere Exceptions fangen kann. Eine `NullPointerException` sollte man jedoch nicht fangen, sondern immer verhindern, dass sie überhaupt auftritt, indem man z.B. überprüft, ob ein Parameter den Wert `null` hat, bevor man eine Methode aufruft.



code20.

Exceptions5c

catch: Behandeln unterschiedlicher Exceptions (2)

- Bei der Behandlung mit try-catch ("Abfangen von Exceptions") kann man in einem catch-Block auch mehrere Exceptions behandeln ("multi-catch"):

```
String fileName = ...;
try {
    printFirst100Chars(fileName);
} catch (NullPointerException | IOException e) {
    System.out.println("Fehlermeldung: " + e.getMessage());
}
```

Das ist praktisch, wenn man unterschiedliche Exceptions genau gleich behandeln möchte.

Disclaimer: Dieses Beispiel illustriert, wie man mehrere Exceptions fangen kann. Eine `NullPointerException` sollte man jedoch nicht fangen, sondern immer verhindern, dass sie überhaupt auftritt, indem man z.B. überprüft, ob ein Parameter den Wert null hat, bevor man eine Methode aufruft.



code20.

Exceptions5d

Exception-Handling: finally

- Bei der Behandlung mit try-catch ("Abfangen von Exceptions") kann man Anweisungen, die auf jeden Fall ausgeführt werden, in einen finally-Block setzen:

```
Scanner scanner = new Scanner(System.in);
try {
    String content =
        Files.readString(Paths.get(scanner.nextLine()));
} catch (IOException e) {
    System.out.println("Fehlermeldung: " + e.getMessage());
} finally {
    scanner.close();
}
```

In diesem Beispiel wird scanner.close() aufgerufen, egal ob eine Exception fliegt oder nicht.



code20.
Exceptions6

Zusammenfassung

- Das Werfen einer Exception beendet den Programmfluss und zeigt an, dass es einen Fehler gab
- Exceptions können über die Methodensignatur an den Methodenaufrufer weitergeleitet...
- ... oder mit try-catch behandelt ("gefangen") werden.
- Man kann verschiedene Exceptions behandeln (mehrere catch-Blöcke oder multi-catch).
- Code im finally-Block wird immer ausgeführt:
 - entweder nach Ende des try-Blocks
 - oder nach Ende der catch-Behandlung
- Es gibt unchecked Exceptions (erben von RuntimeException) und checked Exceptions (erben von Exception, aber nicht RuntimeException)
- Unchecked Exceptions muss man nicht behandeln (kann es aber)

Anwendung: Neu-Eingabe bei Datei-Fehler

Typisches Szenario:

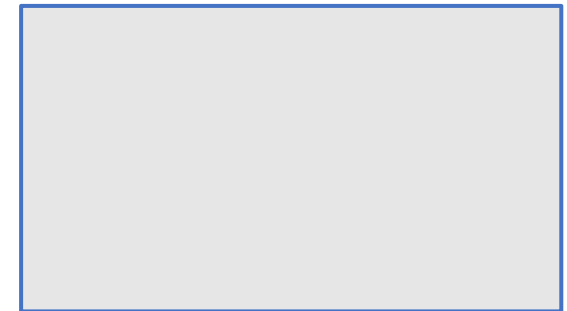
- Nach Benutzereingabe gab es einen Fehler (Exception)
- Eingabe soll wiederholt werden

```
System.out.print("Dateiname? ");
String fileName = scanner.nextLine();

try {
    String content = Files.readString(Paths.get(fileName));
} catch (IOException e) {
    // es gab einen Fehler mit der Datei
    // Wie kann ich einen andere Dateinamen eingeben lassen?
}
```

Siehe Code im
Beispiel!

Alternativ kann man mit `Files.exists(path)` überprüfen, ob eine Datei existiert, bevor man versucht, aus ihr zu lesen.



code20.

Exceptions3
RepeatInput

Anwendung: getter/setter mit Validierung

- Wiederholung: **Attribute sollte man kapseln:**
 - Attribute als `private` deklarieren
 - Lese-Zugriff über öffentliche `getAttribut()`-Methode
 - Schreibzugriff über öffentliche `setAttribut(<neuer Wert>)`-Methode
 - Initiale Werte über den Konstruktor setzen
- An allen Stellen, die Werte setzen, kann man nun überprüfen, ob es sich um einen gültigen Wert handelt
 - setter
 - Konstruktor
- Ansonsten wirft man eine passende `RuntimeException` (unchecked exception)



code20.
gettersetter
(Package)

Komplexeres Beispiel

- Liest das Geburtsjahr des ersten Komponisten aus `composers.csv`
- `getBirthYearOfFirstComposer` und dort aufgerufene Methoden vereinen verschiedene der vorgestellten Konzepte
- siehe Kommentare im Code
- als Nachbereitung ansehen und verstehen



Fehlerbehandlung

In Java: So nicht!

- Funktionsrückgabewerte
 - Anzeigen fehlerhafter Funktionsausführung durch Fehlercodes, typischerweise negative Zahlen
 - Integrierte Fehlerbehandlung

```
public int divide(int a, int b) {  
    if (b != 0) {  
        return a / b;  
    } else {  
        return -2147483648; // = Integer.MIN_VALUE, also kleinster int-Wert  
    }  
}
```

nicht gut!

Integrierte Fehlerbehandlung

- Kein guter Mechanismus, denn:
 - **Verschiedene Rückgabewerte** für verschiedene Methoden (manchmal steht 0 für Fehler, manchmal -1, manchmal jeder negative Wert)
 - Für Methoden wie `divide` auf voriger Folie ist -2147483648 ein gültiger Rückgabewert, z.B. das Ergebnis der Division von $-2147483648 / 1$
 - Man kann also den Fehlerfall nicht vom regulären Fall unterscheiden
 - **Aufrufer muss wissen**, welchen Wert Methode im Fehlerfall liefert
 - Code wird ggf. schwer lesbar, **häufige if-Anweisungen**
 - **Fehler** sollten die **Ausnahme** sein, Rückgabewert einer Methode sollte nicht dem Fehlerfall gewidmet sein
 - **Datentypen** des Rückgabewerts werden **komplexer**, wenn im Wertebereich des Rückgabewerts kein Platz für Fehlerwerte ist

Fehlercode als Referenzparameter

In Java: So nicht!

```
public class ReturnValue {
    public int value;
    public int error;
}

public ReturnValue divide(int a, int b) {
    ReturnValue ret = new ReturnValue();
    if (b == 0) {
        ret.error = -1;
    } else {
        ret.value = a / b;
        ret.error = 0;
    }
    return ret;
}
```

Umständlich, da jetzt alle Methoden, die Fehler haben können, ReturnValue zurückliefern müssen und man den eigentlichen Wert immer erst "auspacken" muss.



code20.

ErrorCodeAs
RefernceParam

Antipattern: String-Rückgabe als Fehlermeldung verwenden

reales Beispiel!

mögliche Ursache:

```
String translate(String key) {  
    String translation =  
        getText(this.currentLocale, key);  
    if (translation == null) {  
        return "translation missing: " + key;  
    } else {  
        return translation;  
    }  
}
```

Schlecht!

Antipattern: Methode

String translate(String key)

gibt **Fehlermeldung** zurück anstatt Übersetzung

Order status

Order number: 3e644e

Updated 2021-05-28 14:21:35 UTC

Order placed Printing Shipped Delivered

(2) XL - Premium Tee

Your order has been shipped.

Your order is en route. It will be delivered by translation missing: en.shipping_provider.PPNT. Your tracking number is UD7746

* Please be advised it may take up to 24 hours for your order to begin showing tracking updates

Fehlerbehandlung mit Exceptions

- An Fehlerstelle mit `throw new <ExceptionTyp>(<Message>)` passende Exception werfen
- `IllegalArgumentException`
 - erbt von `Exception` (über mehrere Sub-Klassen)
 - Im Java SDK vorhandene Exception für ungültige Methodenparameter

```
public int divide(int a, int b) {  
    if (b != 0) {  
        return a / b;  
    } else {  
        throw new IllegalArgumentException("b must not be 0");  
    }  
}
```

im Deutschen: "darf nicht"
(~~nicht "muss nicht"~~)

Fehlerbehandlung mit Exceptions

Üblicher ist folgende Reihenfolge:

- Erst die Parameter auf Gültigkeit überprüfen (ansonsten Exception werfen)
- Dann die eigentliche Berechnung vornehmen

```
public int divide(int a, int b) {  
    if (b == 0) {  
        throw new IllegalArgumentException("b must not be 0");  
    }  
    return a / b;  
}
```

Wichtig: Das else kann man sich hier sparen, da diese Zeile nur dann ausgeführt wird, wenn oben **keine** Exception fliegt, also $b \neq 0$ ist.



code20.
DivisionBy0